



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione

Analisi degli algoritmi e complessità (cenni)

Intelligenza Artificiale

Ingegneria Informatica

Docente

Domenico Fabio Savo

Algoritmo

Algoritmo: Sequenza **finita** di operazioni **non ambigue** ed **effettivamente calcolabili** che, una volta eseguite, producono un risultato in una **quantità finita di tempo**.



algoritmo preparaCaffè

1. Svita la caffettiera.
2. Riempi d'acqua il serbatoio della caffettiera.
3. Inserisci il filtro.
4. Riempi il filtro con la polvere di caffè.
5. Avvita la parte superiore della caffettiera.
6. Metti la caffettiera, così predisposta, su un fornello acceso.
7. Spegni il fornello quando il caffè è pronto.
8. Versa il caffè nella tazzina.

Perché studiare gli algoritmi?

Gli **algoritmi** forniscono il **procedimento** per giungere alla soluzione di un dato **problema di calcolo**

È importante progettare algoritmi che siano:

- **Corretti:** ovvero che producono **correttamente** il risultato desiderato
- **Efficienti:** utilizzino il minor numero di **risorse** in termini di **tempo di esecuzione** ed **occupazione di memoria**



Pseudocodice

Per mantenere il massimo grado di generalità, descriveremo gli algoritmi in **pseudocodice**:

- ricorda linguaggi di programmazione reali come C, Java o Python
- può contenere alcune frasi in linguaggio naturale

La traduzione in un particolare linguaggio di programmazione può essere fatta in modo quasi meccanico



Analisi di un algoritmo

Perché si analizzano gli algoritmi e non i programmi?

- L'analisi teorica (dell'algoritmo) è più affidabile di quella sperimentale (del programma) poiché vale su tutte le possibili istanze di dati su cui l'algoritmo opera
- Ci aiuta a scegliere tra diverse soluzioni allo stesso problema
- Permette di predire le prestazioni di un programma software, prima ancora di scriverne le prime linee di codice



Risorse di calcolo

Per analizzare un algoritmo si misureranno le **risorse di calcolo** da questo utilizzate per la sua esecuzione in **funzione alla dimensione** dell'istanza del problema.

Tali risorse sono:

- **Tempo di esecuzione**
- **Memoria utilizzata (spazio)**

Ad esempio un problema con dimensione n , potrebbe impiegare un tempo $t(n)$ ed uno spazio $s(n)$.



Notazione asintotica

Supponiamo che un algoritmo impieghi $f(n)$ risorse (**tempo di esecuzione** o **occupazione di memoria**) su un input (problema) di dimensione n .

La **notazione asintotica** è un'astrazione utile per descrivere l'ordine di grandezza di $f(n)$ ignorando i dettagli non influenti, come costanti moltiplicative e termini di ordine inferiore.

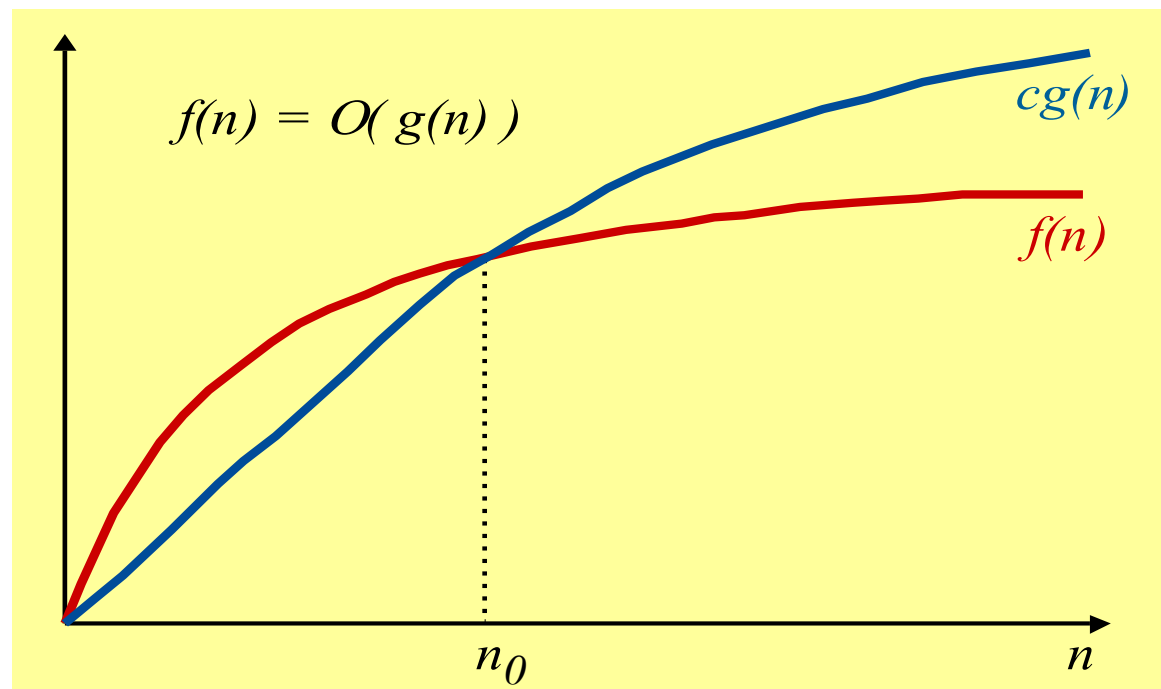


Tre notazioni: O (omicron), Ω (omega), Θ (teta)

- O e Ω forniscono un limite “*lasco*”, rispettivamente per i limiti **superiore** ed **inferiore**
- Θ fornisce un limite “stretto”. In alcuni contesti è difficile trovare un limite stretto per l’andamento delle funzioni, per cui ci si accontenta di un limite meno preciso

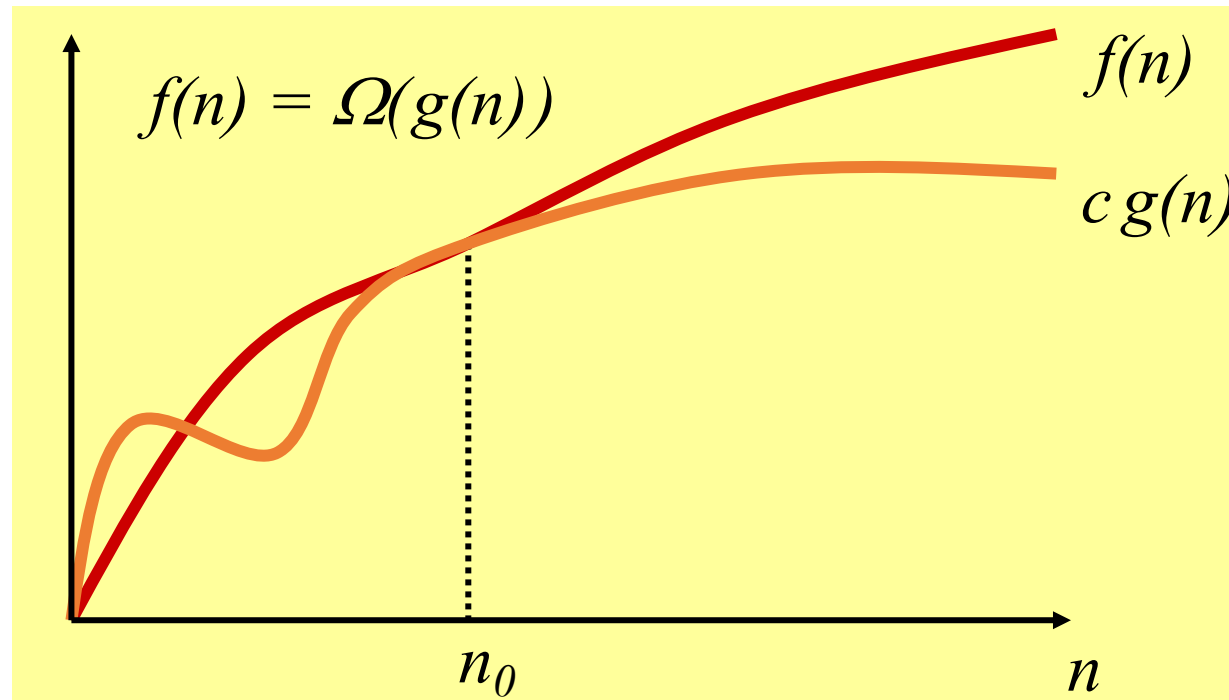
La notazione O

$f(n) = O(g(n))$ se $\exists$ due costanti $c > 0$ e $n_0 \geq 0$ tali che $f(n) \leq c g(n)$ per ogni $n \geq n_0$



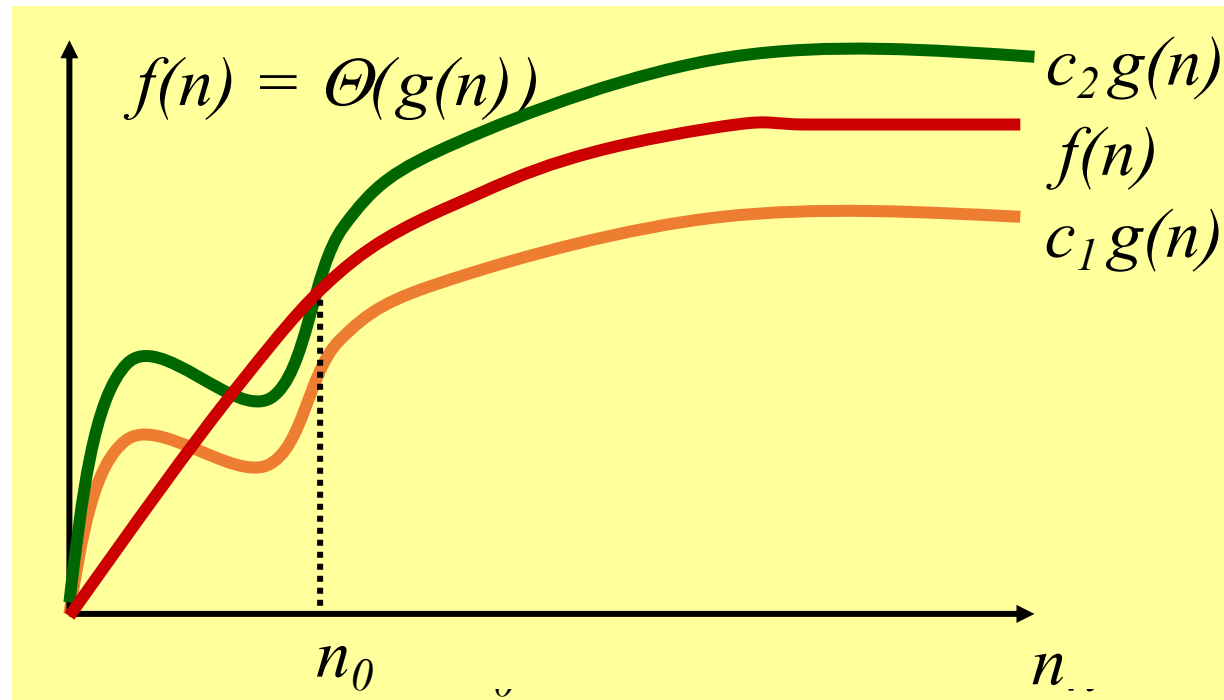
La notazione Ω

$f(n) = \Omega(g(n))$ se $\exists$ due costanti $c > 0$ e $n_0 \geq 0$ tali che $f(n) \geq c g(n)$ per ogni $n \geq n_0$



La notazione Θ

$f(n) = \Theta(g(n))$ se $\exists$ tre costanti $c_1, c_2 > 0$ e $n_0 \geq 0$ tali che
 $c_1 g(n) \leq f(n) \leq c_2 g(n)$ per ogni $n \geq n_0$



Teorema

Dalle definizioni precedenti segue che:

date due funzioni $f(n)$ e $g(n)$, risulta che $f(n) = \Theta(g(n))$ se e solo se

- $f(n) = O(g(n))$
- $f(n) = \Omega(g(n))$

Piccolo abuso di notazione

Si noti che usiamo $f(n) = O(g(n))$ invece di $f(n) \in O(g(n))$

$f(n) = O(g(n))$ si legge $f(n)$ è “**o grande**” di $g(n)$

Se $f(n) = O(g(n))$ è il tempo calcolo richiesto da un algoritmo **Alg** diciamo che $O(g(n))$ è un **limite superiore asintotico** per la complessità tempo di **Alg**

Similmente per le altre notazioni

In questo corso, ci concentreremo maggiormente sulla notazione **O**



Notazione asintotica: esempi

Sia $f(n)=3n^2+10n$

- $f(n)=O(n^2)$: scegliere $c=4$ e $n_0=10$
- $f(n)=\Omega(n^2)$: scegliere $c=1$ e $n_0=0$
- $f(n)=\Theta(n^2)$: infatti $f(n)=\Theta(g(n))$ se e solo se $f(n)=O(g(n))$ e $f(n)=\Omega(g(n))$
- $f(n)=O(n^3)$ ma $f(n)\neq\Theta(n^3)$



Proprietà transitiva di O

La relazione "è O grande di" soddisfa la proprietà transitiva:

Siano $f(n)$, $g(n)$ e $h(n)$ tre funzioni.

Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ allora $f(n) = O(h(n))$



Regole di semplificazione

- 1. Eliminazione delle costanti:** Se $f(n) = O(k g(n))$ con k costante > 0 , allora $f(n) = O(g(n))$
- 2. Somma:** Se $f_1(n) = O(g_1(n))$ e $f_2(n) = O(g_2(n))$, allora $f_1(n) + f_2(n) = O(\max(g_1(n), g_2(n)))$
(Blocchi sequenziali di istruzioni, rami if-then-else, switch)
- 3. Prodotto:** Se $f_1(n) = O(g_1(n))$ e $f_2(n) = O(g_2(n))$, allora $f_1(n)f_2(n) = O(g_1(n)g_2(n))$
(Cicli for, while, do-while)

Un po' di terminologia

Alcune *classi di complessità asintotica*:

- **$O(1)$: costante**, non dipende dalla dimensione dei dati
- **$O(\lg n)$: logaritmica** (in generale: $\lg^k n$ $k \geq 1$)
- **$O(n)$: lineare**
- **$O(n \lg n)$: pseudolineare**
- **$O(n^2)$: quadratica** (n^k polinomiale $k > 1$)
- **$O(2^n)$: esponenziale** (in generale: k^n $k > 1$)



Complessità intrinseca di un problema

Complessità intrinseca di un problema
 $\neq$
Complessità computazionale di un algoritmo

Definizioni:

Un **problema computazionale** ha **complessità $O(f(n))$** (upper bound) se esiste **un** algoritmo per la sua risoluzione con delimitazione superiore $O(f(n))$.

Un **problema computazionale** ha **complessità $\Omega(f(n))$** (lower bound) se **tutti** gli algoritmi per la sua risoluzione hanno delimitazione inferiore $\Omega(f(n))$.



Algoritmi ottimali

Se dimostro che un **problema** ha **delimitazione inferiore** $\Omega(f(n))$ e trovo un **algoritmo** **avente delimitazione superiore** $O(f(n))$ allora, a meno di costanti, ho un **algoritmo ottimale** per risolvere il problema!!!

Esempio (Ricerca del minimo in un insieme non ordinato):

Un algoritmo per la ricerca del minimo in un insieme non ordinato, avente complessità $O(n)$, è ottimo. Infatti, ogni algoritmo dovrà almeno leggere l'input, e quindi il problema avrà complessità inferiore $\Omega(n)$.



Complessità computazionale

La **complessità computazionale** ha l'obiettivo di studiare quanto **difficile** sia risolvere uno specifico problema.

La teoria della complessità studia i **problema di decisione**: cioè, problemi che ammettono solo **si** o **no** come soluzione.

Un **algoritmo di decisione** per un problema è un algoritmo che calcola il corretto valore di verità per ogni possibile istanza del problema (**l'algoritmo deve terminare per tutti gli input**):

- **input**: un'istanza del problema
- **output**: **si** o **no**

Un **problema di decisione** si dice **decidibile** se esiste almeno un algoritmo di decisione per esso. Altrimenti è detto **indecidibile**.



Funzione di complessità

La complessità si misura in termini di quantità di risorse (**tempo**, **spazio**) di cui l'algoritmo ha bisogno per risolvere il problema.

Per misurare la complessità del problema, si considera il **miglior algoritmo possibile** che lo risolve.

Analisi della complessità nel **caso peggiore**:

la complessità è misurata in termini di una funzione (di complessità) **f**:

- **argomento**: la dimensione **n** di un'istanza del problema
- **risultato**: la quantità **f(n)** di tempo/spazio necessaria, nel **caso peggiore**, per risolvere un'istanza di dimensione **n**.



Classe di complessità (1\2)

Per astrarre dalle questioni contingenti (ad esempio, il linguaggio di programmazione, la velocità del processore e così via), facciamo riferimento a un modello astratto di calcolo: **le macchine di Turing (TM)**.

Di solito non si considerano specifiche funzioni di complessità **f**, ma piuttosto famiglie **C** di funzioni di complessità, che danno origine a **classi di complessità**.

Definizione: Una classe di complessità tempo/spazio **C** è l'insieme di tutti i problemi **P** tali che un'istanza di **P** di dimensione **n** può essere risolta in un tempo/spazio al massimo **C(n)**.



Classe di complessità (2\2)

Definizione: Classi **TIME**($f(n)$) e **SPACE**($f(n)$).

Data una qualunque funzione $f(n)$ non negativa, chiamiamo **TIME**($f(n)$) e **SPACE**($f(n)$) gli **insiemi dei problemi decisionali** che possono essere risolti rispettivamente in tempo e spazio $O(f(n))$.

Esempio: il problema di verificare se un certo elemento è presente in un dizionario di n elementi appartiene alle classi **TIME**($\log(n)$) e **SPACE**($\log(n)$).



La classe PTime (P)

P è la classe dei problemi che possono essere risolti da **algoritmi polinomiali deterministici**.

I problemi in questa classe sono detti **problemi trattabili**, cioè risolvibili per input di grandi dimensioni.

Sfortunatamente, per molti problemi di interesse pratico non sono mai stati trovati algoritmi polinomiali e si presume che non esistano affatto



La classe NPTime (NP)

La classe dei problemi che possono essere risolti da algoritmi **polinomiali non-deterministici**.

In un modello di calcolo non-deterministico (ad es. la **MT non-deterministica**) una computazione può diramarsi in un albero di computazioni che si sviluppano indipendentemente.

Questi problemi sono considerati **intrattabili**, cioè non risolvibili in "tempo ragionevole" per input di grandi dimensioni.

I migliori algoritmi conosciuti per risolvere questo tipo di problemi richiedono infatti **tempo esponenziale**.

Corrispondo ad una larga classe di problemi pratici per i quali il seguente tipo di algoritmo può essere usato:

1. Indovina in modo non-deterministico una possibile soluzione di dimensione polinomiale.
2. Verifica in tempo polinomiale se la soluzione indovinata è buona



La questione **P** vs **NP**

$P \neq NP?$

Uno dei *problemi del millennio*

- <https://www.claymath.org/millennium-problems/>

Congettura di cui non è disponibile per il momento alcuna dimostrazione, si suppone però che **P** $\neq$ **NP**



Il complemento dei problemi NP: coNP

coNP è l'insieme dei problemi il cui complemento è in **NP**, cioè i problemi per i quali determinare se un'istanza ammette una risposta **negativa** è in **NP**.

Per i problemi la cui complessità è caratterizzata in termini di **MT non-deterministica**, la soluzione del problema e la soluzione del suo complemento possono infatti essere diverse.

La ragione di ciò è che una risposta affermativa viene restituita se esiste un percorso di calcolo non-deterministico della MT che porta all'accettazione. Una risposta negativa richiede invece che tutti i percorsi di calcolo non-deterministici della MT portino al rifiuto.

Si ritiene che **coNP** sia diverso sia da **NP** che da **P**.



Classi di complessità superiori a NP

Definizione: **PSpace** è l'insieme dei problemi risolvibili in uno **spazio** polinomiale da una MT deterministica. Lo spazio polinomiale non è "veramente buono", poiché questi problemi possono richiedere **tempi esponenziali**. In effetti, si ritiene che questi problemi siano più difficili dei problemi in **NP**.

Definizione: **ExpTime** è l'insieme dei problemi risolvibili in **tempo** esponenziale da una MT deterministica. Questi problemi sono considerati molto difficili.

Definizione: **NExpTime** è l'insieme dei problemi risolvibili in **tempo** esponenziale da un MT non-deterministica.



Classi di complessità inferiori a P

Definizione: **LogSpace** (**NLogSpace**) è l'insieme dei problemi risolvibili nello **spazio** logaritmico da una MT (non) deterministica. Nota: quando si misura la complessità spaziale, la dimensione dell'input non conta e si considera solo la memoria di lavoro.

Definizione: **AC0** è l'insieme dei problemi risolvibili in **tempo** costante utilizzando un numero polinomiale di processori. Questi problemi sono risolvibili in modo efficiente anche per input molto grandi.



Ringraziamenti

Ringrazio la Prof.ssa Scandurra per avermi fornito il materiale da cui è stato estratto la maggior parte del contenuto di queste slide.



**UNIVERSITÀ
DEGLI STUDI
DI BERGAMO**

Dipartimento
di Ingegneria Gestionale,
dell'Informazione e della Produzione